

Bug Severity Classification System

High Level Design Document

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Scope	2
1.3	Intended Audience	2
1.4	Definitions and Abbreviations	2
2	System Overview	2
3	Architecture Overview	3
3.1	Component Summary	3
4	Component Design	3
4.1	FastAPI Backend	3
4.2	Machine Learning Pipeline	4
4.3	Drift Detection	4
4.4	Airflow Orchestration	4
4.5	Monitoring Stack	5
4.5.1	Prometheus	5
4.5.2	Grafana	5
4.5.3	AlertManager	5
5	Data Flow	5
5.1	Prediction Request Flow	5
5.2	Retraining Flow	6
6	Deployment Architecture	6
6.1	Docker Compose	6
6.2	Service Dependencies	6
7	Security Considerations	6
8	Scalability and Limitations	7
8.1	Current Limitations	7
9	Summary of Design Choices	7

1 Introduction

1.1 Purpose

This document provides a high-level design overview of the Bug Severity Classification System — an end-to-end MLOps application that automatically predicts the severity of software bugs from textual summaries. It is intended to serve as a reference for understanding the system's architecture, component responsibilities, data flow, and key design decisions.

1.2 Scope

The system encompasses:

- A REST API backend for serving predictions and managing users
- A machine learning pipeline for preprocessing, training, and evaluating classification models
- Experiment monitoring the experiments using MLFlow
- An automated orchestration pipeline using Apache Airflow
- A drift detection mechanism to trigger automatic retraining
- A monitoring and alerting stack using Prometheus, Grafana, and AlertManager
- A fully containerized deployment using Docker and Docker Compose

1.3 Intended Audience

- Software engineers and MLOps engineers
- Quality Analysis Tester

1.4 Definitions and Abbreviations

Term	Definition
API	Application Programming Interface
ML	Machine Learning
DAG	Directed Acyclic Graph
LR	Logistic Regression
RF	Random Forest
KS Test	Kolmogorov-Smirnov statistical test
MLflow	ML experiment tracking platform
JWT	JSON Web Token (authentication mechanism)

2 System Overview

The Bug Severity Classification System is a production-grade MLOps application designed to assist software development teams in triaging bugs efficiently. Given a plain-

text bug summary, the system predicts one of five severity levels: *trivial*, *minor*, *normal*, *major*, or *critical*. From the predictions it can be inferred which task has to be prioritized.

3 Architecture Overview

3.1 Component Summary

The system is composed of eight major components, all containerized and communicating over a shared Docker network (`bugs_network`).

Component	Port	Responsibility
FastAPI Backend	8000	Serves predictions, manages users, exposes Prometheus metrics
PostgreSQL	5432	Stores users, predictions, and feedback
Apache Airflow	8080	Orchestrates the ML pipeline DAG
MLflow	5000	Tracks experiments, metrics, and model versions
Prometheus	9090	Scrapes and stores time-series metrics
Grafana	3000	Visualizes metrics via dashboards
AlertManager	9093	Routes alerts to email and fires drift alerts
Node Exporter	9100	Exposes system-level CPU, RAM, and disk metrics

4 Component Design

4.1 FastAPI Backend

The backend is the central hub of the system. It is responsible for:

- Authenticating users using JWT tokens with bcrypt password hashing
- Accepting bug summaries and returning severity predictions
- Recording predictions and user feedback to PostgreSQL
- Exposing a `/metrics` endpoint scraped by Prometheus
- Serving the frontend HTML pages as static files

Design Choice — FastAPI over Flask: FastAPI was chosen for its native asynchronous support, and Pydantic-based request validation.

4.2 Machine Learning Pipeline

The ML pipeline consists of four sequential scripts managed by DVC and orchestrated by Airflow:

Stage	Script	Output
Preprocess	preprocess.py	train.csv, val.csv, test.csv
Transform	transform.py	train.parquet, val.parquet, test.parquet
Train	train.py	best_model.pkl, best_model_metrics.json
Evaluate	evaluate.py	test_metrics.json

Design Choice — Two Models: Logistic Regression and Random Forest were selected as complementary models. LR is fast, interpretable, and performs well on linearly separable data. RF handles non-linear patterns and feature interactions. The model with the higher weighted F1 score on the validation set is automatically selected as the best model.

Design Choice — DVC for Pipeline Versioning (CI/CD): DVC tracks data and model artifacts, ensuring reproducibility across runs. Each pipeline stage declares explicit inputs and outputs, preventing unnecessary re-execution of unchanged stages.

4.3 Drift Detection

Drift is detected through two independent checks run by `drift_detection.py`:

1. **Feedback Accuracy Drop:** The system queries the last 100 predictions with user feedback from PostgreSQL. If the proportion of correctly predicted labels falls below 95%, performance drift is flagged.
2. **Kolmogorov-Smirnov (KS) Test:** The distribution of predicted severity labels from recent production data is compared against the distribution from training data (stored in `metrics/train_distribution.json`). A p-value below 0.05 indicates a statistically significant distributional shift.

Drift is flagged if **either** check fails, triggering the retraining pipeline.

Design Choice — KS Test over PSI: The KS test was chosen for its non-parametric nature and simplicity. It makes no assumptions about the underlying distribution, making it appropriate for the ordinal severity labels used in this system.

4.4 Airflow Orchestration

The Airflow DAG (`bugs_severity_pipeline`) implements a branching workflow:

- **First Run:** If no model exists, the full training pipeline runs unconditionally.
- **Subsequent Runs:** If a model exists, production data is loaded from the DB. If the DB is empty, the pipeline skips gracefully. If drift is detected, feedback data is appended to the training CSV, and the full retraining pipeline is triggered.
- **Email Notification:** Gmail SMTP is used to send success, failure, and skip notifications after every DAG run.

- **Prometheus Alert:** On drift detection, a custom alert is fired directly to Alert-Manager via its REST API.

Design Choice — Schedule + Manual Trigger: The DAG runs daily by default but can be manually triggered from the Airflow UI. This gives operators flexibility to retrain on demand without waiting for the schedule.

4.5 Monitoring Stack

4.5.1 Prometheus

Prometheus scrapes two targets every 15 seconds:

- `bugs_app` at `localhost:8000/metrics` — application-level metrics
- `node_exporter` at `localhost:9100/metrics` — system-level metrics

Four metric types are instrumented: Counters (total predictions, logins, errors), Gauges (active requests, DB record counts), Histograms (prediction and login latency), and Summaries (confidence score distribution).

4.5.2 Grafana

A 14-panel dashboard provides real-time visibility into:

- Application health (UP/DOWN), throughput, and latency percentiles (p50, p95, p99)
- Prediction distribution by severity
- Login success vs failure rates
- Feedback accuracy trends
- System CPU, RAM, and disk utilization

4.5.3 AlertManager

AlertManager routes alerts via Gmail SMTP. Nine alerting rules are defined covering: application downtime, high login failure rates, high HTTP error rates, CPU/RAM/disk overuse, prediction latency anomalies, and model drift.

5 Data Flow ---

5.1 Prediction Request Flow

1. User submits a bug summary via the frontend
2. FastAPI authenticates the JWT token
3. The bug summary is passed to the loaded ML model
4. The model returns a severity label and confidence scores
5. The prediction is stored in PostgreSQL
6. Prometheus metrics are updated

7. The result is returned to the user

5.2 Retraining Flow

1. Airflow DAG triggers (scheduled or manual)
2. Raw CSV data is verified
3. Model existence is checked
4. Production predictions are loaded from PostgreSQL
5. Drift detection runs (accuracy check + KS test)
6. If drift detected: Prometheus alert fired, feedback rows appended to CSV
7. Full pipeline runs: preprocess → transform → train → evaluate → deploy
8. Email notification sent with metrics

6 Deployment Architecture ---

6.1 Docker Compose

All services are containerized and run on a single host using Docker Compose. Services communicate over a shared bridge network (**bugs_network**). Persistent data is stored in named Docker volumes for PostgreSQL and MLflow artifacts. .

6.2 Service Dependencies

- FastAPI depends on PostgreSQL being healthy
- Airflow depends on PostgreSQL (metadata DB) and the ML model artifacts volume
- Prometheus depends on FastAPI and Node Exporter being reachable
- Grafana depends on Prometheus as its data source
- AlertManager is configured as Prometheus's notification receiver

7 Security Considerations ---

- **Authentication:** All prediction and history endpoints require a valid JWT token. Tokens are signed with a secret key using the HS256 algorithm.
- **Password Storage:** User passwords are hashed using bcrypt before storage. Plain-text passwords are never stored.
- **Role-Based Access:** The system supports user roles (admin, regular user). Admin endpoints are separated from user endpoints.

8 Scalability and Limitations

8.1 Current Limitations

- The system is deployed on a single machine. There is no horizontal scaling of the API.
- Drift detection requires a minimum of 10 feedback records and 20 production predictions before checks activate.
- The ML model is loaded from disk on each prediction (no in-memory model server).
- MySQL is provisioned but not actively used in the current implementation.

9 Summary of Design Choices

Decision	Choice	Rationale
API Framework	FastAPI	Async support, auto docs, type safety
Database	PostgreSQL	ACID compliance, JSON column support, robust for relational data
ML Models	LR + RF	Complementary strengths; best model auto-selected by F1 score
Pipeline Versioning	DVC	Reproducible pipelines, artifact tracking
Experiment Tracking	MLflow	Metric logging, model registry, run comparison
Orchestration	Apache Airflow	DAG-based workflows, scheduling, branching logic
Drift Detection	KS Test + Accuracy	Non-parametric, no distribution assumptions, interpretable
Monitoring	Prometheus	Pull-based, time-series, widely supported
Visualization	Grafana	Rich dashboards, Prometheus native integration
Alerting	AlertManager	Grouping, silencing, email routing
Containerization	Docker Compose	Simple single-host deployment, low overhead
Authentication	JWT + bcrypt	Stateless, secure, industry standard